JSS2 DIGITAL TECHNOLOGIES LESSON NOTES (SECOND TERM)

WEEK 1: Introduction to Programming

Topic: Introduction to Programming

Subtopic 1: What is Programming?

- **Program:** A program is a set of step-by-step instructions, written in a special language, that tells a computer exactly what to do. Think of it like a perfect recipe for a computer to follow.
- **Programming (or Coding):** This is the process of writing, testing, and fixing these instructions. The person who does this is called a **Programmer** or a **Coder**.
- **Analogy:** A computer is like a very smart but "dumb" robot. It will only do exactly what you tell it. Programming is the act of writing the perfect recipe (the program) so the robot can perform a task.

Subtopic 2: Importance of Programming

1. **It Powers All Technology:** Every piece of software you use – from video games and apps to Windows and Google Chrome – is a program. Without programming, computers are useless.
2. **Problem Solving:** Programming teaches you how to think logically and break down big, difficult problems into small, simple steps that a computer can solve.

3. **Automation:** It allows us to make computers do boring, repetitive tasks for us, like calculating the grades for every student in a school.
4. **Career Opportunities:** Knowing how to program is one of the most valuable skills in the world, leading to jobs like Software Engineer or Web Developer.

Subtopic 3: Types of Programming Languages

A programming language is the special vocabulary and set of rules used to write instructions.

- **A. Low-Level Languages:** These are languages very close to the computer's own language (binary: 0s and 1s). They are very hard for humans to read and write but are very fast for the computer.
  - *Examples:* Machine Language (just 0s and 1s), Assembly Language (uses short codes like `ADD`).
- **B. High-Level Languages:** These are closer to human language (like English). They are much easier for humans to read, write, and understand.
  - *Examples:* Python, Java, C++, Scratch.

Subtopic 4: Block-Based vs. Text-Based Programming (Both are High-Level)

- **Block-Based (Visual) Programming:** You build a program by dragging and dropping graphical blocks, like "digital Lego bricks." It's great for beginners because you can't make typing errors.
  - *Example:* Scratch. To make a cat move, you drag a `move 10 steps` block.
- **Text-Based Programming:** You build a program by typing all the commands and instructions using precise keywords and punctuation (this is called **syntax**). If you make a typing mistake, the program will show an error.
  - *Example:* Python. To show a message, you must type: `print("Hello, World!")`

Evaluation Questions:

1. What is a "program"?

2. What is "programming"?
3. State two reasons why programming is important.
4. What is the main difference between a high-level and a low-level language?
5. Give one example of a block-based language and one example of a text-based language.

WEEK 2: Basic Algorithms

Topic: Basic Algorithms

Subtopic 1: What is an Algorithm?

- **Definition:** An algorithm is a clear, step-by-step set of instructions designed to perform a specific task or solve a problem. It's the plan you make before you start writing the actual code.
- **Analogy:** It is the "recipe" for solving a problem.
- **Characteristics:** A good algorithm must be:
  - **Clear (Unambiguous):** Each step must have only one meaning.
  - **Finite:** It must have an end; it can't go on forever.
  - **Correct:** It must actually solve the problem.

Real-Life Examples:

- **Making Tea:** 1. Get a cup. 2. Put a teabag in. 3. Boil water. 4. Pour water into cup. 5. Add milk/sugar. 6. Stir.
- **Brushing Teeth:** 1. Get brush. 2. Apply toothpaste. 3. Brush all surfaces of teeth. 4. Rinse mouth.

Subtopic 2: Steps in Problem Solving (Programming)

You don't just start coding. You must plan first.

PETOA GROUP OF SCHOOLS

1. **Define the Problem:** Understand exactly what you need to do. (e.g., "I need to find the average of three numbers.")
2. **Plan the Solution (Algorithm):** Design the step-by-step instructions. This is where you use tools like flowcharts or pseudocode.
3. **Code (Write the Program):** Translate your algorithm into a programming language (like Scratch or Python).
4. **Test and Debug:** Run the program. Does it give the right answer? If not, find the errors ("bugs") and fix them ("debugging").
5. **Document:** Write comments in your code to explain it, so others can understand it later.

Subtopic 3: Planning Tools: Flowcharts

A **flowchart** is a diagram that represents an algorithm, using standard symbols connected by arrows to show the flow of logic.

Basic Flowchart Symbols:

- **Oval (Terminator):** Represents the **START** or **END** of an algorithm.
- **Parallelogram (Input/Output):** Used for getting data (e.g., "Get A") or showing results (e.g., "Display Sum").
- **Rectangle (Process):** Represents a calculation or an action (e.g., `Sum = A + B`).
- **Diamond (Decision):** Used to ask a "Yes/No" question (for conditions). (e.g., `Is A > B?`).
- **Arrows (Flow Lines):** Show the direction of the steps.

Subtopic 4: Planning Tools: Pseudocode

Pseudocode means "fake code." It is a way of writing your algorithm using a simple, English-like structure. It has no strict rules, making it easy to write and read.

Example: Algorithm to find the sum of two numbers.

- **Pseudocode:**

1. START
2. Get first number (A)

3. Get second number (B)
4. Calculate Sum = A + B
5. Display Sum
6. END

Evaluation Questions:

1. What is an algorithm?
2. Give one real-life example of an algorithm.
3. What is a flowchart?
4. What is pseudocode?
5. What shape is used for "START" and "END" in a flowchart? What shape is used for a calculation?

WEEK 3: Block Coding

Topic: Block Coding with Scratch

Subtopic 1: What is Block-Based Coding?
Block-based coding uses graphical "blocks" that snap together like puzzle pieces. This lets you focus on the logic of your program (the algorithm) without worrying about making typing mistakes. Our main tool for this is **Scratch**.

Subtopic 2: The Scratch Interface
When you open Scratch, you'll see four main areas:

1. **The Stage (Top-Left):** This is the "screen" where your program, game, or animation runs. The green flag (to start) and red stop sign (to stop) are here.
2. **The Sprite List (Bottom-Left):**

- **Sprite:** A sprite is a character or object in your program (like the default orange cat).

- This area shows all the sprites in your project.

3. **The Block Palette (Middle):** This is your "toolbox." It contains all the code blocks, organized by color and function.

- **Motion (blue):** `move`, `turn`, `go to...`
- **Looks (purple):** `say`, `think`, `change costume`
- **Sound (magenta):** `play sound...`
- **Events (yellow):** `when green flag clicked`, `when space key pressed...`
- **Control (orange):** `wait`, `repeat`, `if...then...`

4. **The Script Area (Right):** This is your "workspace." You drag blocks from the palette and snap them together here to build your program, which is called a **script**.

Subtopic 3: What is a Script?

A **script** is a stack of blocks that are connected together. Each sprite has its own script area and can have its own scripts to control what it does.

Subtopic 4: Creating a Basic Animation (Your First Script)

Goal: Make the cat sprite move, say "Hello!", and turn.

1. From the **Events** (yellow) palette, drag a `when green flag clicked` block into the Script Area. This will start your script.
2. From the **Motion** (blue) palette, drag a `move 10 steps` block and snap it under the first block. Change "10" to "100".
3. From the **Looks** (purple) palette, drag a `say "Hello!" for 2 seconds` block and snap it under the "move" block.
4. From the **Motion** (blue) palette, drag a `turn (clockwise) 15 degrees` block and snap it at the bottom. Change "15" to "90".

Now, click the green flag on the Stage. The cat will move, say "Hello!", and turn. This is a simple animation!

Evaluation Questions:

1. What is a "sprite" in Scratch?
2. What is the "stage"?
3. What is the "block palette"?
4. What is a "script"?
5. Which Events block is most commonly used to start a script?

WEEK 4: Variables in Programming

Topic: Variables in Programming

Subtopic 1: What is a Variable?

- **Definition:** A variable is a placeholder in a computer's memory used to store a piece of information. The information it holds (its "value") can change or vary while the program is running.
- **Analogy:** Think of a variable as a labelled box or container.
  - The **name** of the variable is the label on the box (e.g., "score").
  - The **value** of the variable is the item inside the box (e.g., the number 10).

Subtopic 2: Uses of Variables

We use variables to store data that our program needs, such as:

- A player's score in a game.
- The number of lives a player has left.
- A player's name.
- The speed of a character.
- The answer to a math question.

Subtopic 3: Creating and Naming Variables in Scratch

PETOA GROUP OF SCHOOLS

1. Go to the **Variables** (dark orange) palette.
2. Click the "Make a Variable" button.
3. A box will appear. Type a name for your variable, like `score`.
4. Choose "For all sprites" (for now) and click OK.
5. Your new variable will appear in the palette with its own special blocks, and a monitor for it will appear on the Stage, showing its name and current value.

Subtopic 4: Using Variables (Example: A Scorekeeper)

Goal: Create a program where the score starts at 0 and increases by 1 every time you click the cat sprite.

- **Script 1 (To set up the score at the start):**

1. `when green flag clicked` (from Events)
2. `set [score] to (0)` (from Variables)

- **Script 2 (The action when the cat is clicked):**

1. `when this sprite clicked` (from Events)
2. `change [score] by (1)` (from Variables)

Now, every time you click the cat, the score on the stage will go up by 1.

Evaluation Questions:

1. What is a variable? Use the "box" analogy.
2. What is the "value" of a variable?
3. List two things we use variables for in a game.
4. In Scratch, what colour is the Variables palette?
5. What is the difference between the `set [score] to (0)` block and the `change [score] by (1)` block?

PETOA GROUP OF SCHOOLS

WEEK 6: Control with Conditions

Topic: Control with Conditions

Subtopic 1: What is a Conditional Statement?

- **Definition:** A conditional statement (or "condition") allows a program to make decisions. It checks if a certain condition is **True** or **False**, and then performs different actions based on the result.
- **Analogy:**
  - "IF it is raining, THEN I will take an umbrella." (A simple condition)
  - "IF it is raining, THEN I will take an umbrella, ELSE I will wear my sunglasses." (A two-part condition)

Subtopic 2: Conditions in Scratch (Sensing)

To make a decision, you first need to check something. The **Sensing** (light blue) palette contains blocks that ask "True/False" questions, like `touching (mouse-pointer)?`.

Subtopic 3: The "IF...THEN" Block

- **Scratch Block:** `if < > then`
- **Function:** This is a one-part decision. If the condition in the hexagonal slot is **TRUE**, it will run the blocks inside the "C" shape. If it is **FALSE**, it skips those blocks.
- **Example Script:** Make the cat hide when it touches the mouse pointer.

```text

when green flag clicked
forever
    if <touching (mouse-pointer)?> then
        hide
    end
end
```

The `forever` loop keeps checking. The moment the mouse touches the cat (condition becomes TRUE), it runs the `hide` block.

Subtopic 4: The "IF...THEN...ELSE" Block

- **Scratch Block:** `if < > then` ... `else`
- **Function:** This is a two-part decision. It always does one of two things.
  - If the condition is **TRUE**, it runs the blocks in the first "C" shape.
  - If the condition is **FALSE**, it runs the blocks in the second "C" shape (the `else` part).
- **Example Script:** Make the cat say "Ouch!" if it's touching the wall, or "Safe!" if it's not.

```text
when green flag clicked
forever
    if <touching (edge)?> then
        say "Ouch!" for (0.5) seconds
    else
        say "I am safe!" for (0.5) seconds
    end
end
```

Evaluation Questions:

1. What is a conditional statement?
2. What is the difference between an `if...then` and an `if...then...else` block?
3. What shape are the condition blocks from the Sensing palette?
4. In an `if...then...else` block, when does the "else" part run?
5. What Sensing block would you use to check if the space bar is being pressed?

WEEK 8: Loops in Programming

Topic: Loops in Programming

Subtopic 1: What is a Loop?

- **Definition:** A loop (or iteration) is a control structure that allows you to repeat a set of instructions multiple times. Loops are fundamental because they save you time and make your code efficient.
- **Analogy (Without a loop):** To make a sprite move in a square, you'd need 8 blocks: move, turn, move, turn, move, turn, move, turn.
- **Analogy (With a loop):** You can just say: Repeat 4 times { move 100 steps; turn 90 degrees; }. This is only 3 blocks, cleaner, and easier to change.

Subtopic 2: Types of Loops

- **A. "FOR" Loop (Definite Loop):**
  - Repeats a block of code a fixed, known number of times.
  - You know exactly how many times it will run before it starts.
  - **Scratch Block:** `repeat (10)` (from the Control palette).
  - **Use:** For drawing a square (`repeat (4)`), or making a sprite say something 5 times.
- **B. "WHILE" Loop (Indefinite Loop):**
  - Repeats as long as (or until) a certain condition is true.
  - You do not know how many times it will run in advance.
  - **Scratch Blocks:**
    - `forever` (This is a "while TRUE" loop. It never stops).
    - `repeat until < >` (This repeats until the condition becomes TRUE, then it stops).
  - **Use:** For keeping a game running (`forever`), or moving a character until it hits a wall (`repeat until <touching (edge)?>`).

Example Script (FOR Loop for a square):

```
text
when green flag clicked
repeat (4)
    move (100) steps
    turn (90) degrees
end
wait (1) seconds
```

Example Script (WHILE Loop - move until the wall):

```text
when green flag clicked
go to x: (-200) y: (0)
repeat until <touching (edge)?>
    move (10) steps
end
say "I'm at the edge!" for (2) seconds
```

Evaluation Questions:

1. What is a "loop" in programming?
2. Why are loops so important? (What do they save?)
3. What is a "FOR" loop (or definite loop)?
4. What is a "WHILE" loop (or indefinite loop)?
5. Which Scratch block is a "FOR" loop and which is a "WHILE" loop?

PETOA GROUP OF SCHOOLS

WEEK 9: Game Development with Visual Programming

Topic: Game Development with Visual Programming

Subtopic 1: Steps in Game Creation (Game Design Process)

1. **Brainstorm (Plan):** What is your game about? What is the goal? (e.g., a "chase" game where the cat must catch the mouse).
2. **Design Assets:** Who are the characters (sprites)? What does the background (stage) look like?
3. **Build Scripts (Code):** Add the blocks. How does the player move? How do you score? How do you win?
4. **Test and Debug:** Play the game. Does it work? Is it fun? Find and fix any problems (bugs).

5. **Refine:** Make it better by adding sounds, new levels, or improving the graphics.

Subtopic 2: Designing a Simple Game: "Cat and Mouse"

- **Goal:** The player controls the Cat to catch the Mouse.
- **Sprites:** A Cat and a Mouse.
- **Stage:** Any background you like.

Subtopic 3: Building the Scripts (Combining Our Skills)

- **Script for the Cat (Player Control):** We want the player to control the cat with the mouse pointer.

```text

when green flag clicked
forever
    go to (mouse-pointer)
end
```

- **Script for the Mouse (The Target):** We want the mouse to check if it's being caught.

```text

when green flag clicked
show
forever
    if <touching (Cat)?> then
        say "You caught me!" for (2) seconds
        hide
        stop [all]
    end
end
```

Putting it all together: When you click the green flag, you can fly the cat around with your mouse. The moment the cat touches the mouse, the mouse will say "You caught me!", hide, and the game will end. You have made an interactive game!

Evaluation Questions:

1. List the first three steps in game creation.
2. In our "Cat and Mouse" game, how did we make the Cat interactive?
3. What Sensing block did the mouse use to check for the cat?
4. What Loop block did both sprites use?
5. What was the "win condition" in our game?

WEEK 10: Debugging

Topic: Debugging

Subtopic 1: Meaning of "Bug" and "Debugging"

- **Bug:** A bug is an error, mistake, or fault in a computer program that causes it to behave in unexpected ways, give the wrong answer, or crash.
- **Debugging:** Debugging is the process of finding and fixing these bugs.

Fun Fact: The term "bug" was famously used in the 1940s when a real moth (a bug!) got stuck inside a computer and caused it to stop working.

Subtopic 2: Importance of Debugging
ALL programmers, from beginners to experts, make mistakes. Being a good programmer is not about never making bugs, it's about being good at **finding and fixing them**. Debugging is a normal and essential part of programming.

Subtopic 3: Types of Bugs (Errors)

- **Syntax Errors:** A typing or grammar error in a text-based language (like misspelling `print` as `prnt`). The program won't even run. (These don't happen in block-based coding).
- **Logic Errors:** The program runs, but it doesn't do what you wanted it to do. The logic of your algorithm is wrong. These are common in Scratch.

Example of a Logic Error (and how to fix it):

- **The Goal:** You make the "Cat and Mouse" game from Week 9.
- **The Bug:** You click the green flag, and the score immediately goes to 1 and the mouse says "You caught me!" before you even move the cat.
- **The Problem (Logic Error):** You forgot to move the Cat and Mouse apart at the start. They are touching each other, so the `touching (Cat)?` condition is TRUE immediately.
- **The Fix:** Add `go to x: (100) y: (100)` to the Mouse's start script, and `go to x: (-100) y: (-100)` to the Cat's start script, so they start on opposite sides.

Subtopic 4: Debugging Techniques

1. **Use "Say" Blocks:** If you don't know what value a variable has, add a `say (score)` block inside your loop. The sprite will "say" its score, and you can see if it's changing correctly.
2. **Test One Block at a Time:** If a long script isn't working, try clicking on each block individually to see what it actually does.
3. **Read Your Code Slowly:** Go through your script, block by block, and pretend you are the computer. Does what it does match what you wanted it to do?
4. **Ask a Friend:** Sometimes a fresh pair of eyes can spot a mistake you've missed.

Evaluation Questions:

1. What is a "bug" in programming?
2. What is "debugging"?
3. Why is debugging a normal part of programming for everyone?
4. What is a "logic error"?
5. What is one technique you can use to find a bug in a Scratch script?

WEEK 11: Revision

Topic: Revision of Programming Concepts

This week is for reviewing all the key programming concepts we have learned this term.

Summary of Key Concepts:

- **Week 1 (Programming):** A program is a set of instructions for a computer. We can use block-based (Scratch) or text-based (Python) languages.
- **Week 2 (Algorithms):** An algorithm is the step-by-step plan for your program. We use flowcharts (diagrams) and pseudocode (fake code) to plan.
- **Week 4 (Variables):** A variable is a "box" in the computer's memory used to store data that can change, like a player's score.
- **Week 6 (Conditions):** Conditions are "decision-making" structures (like `if...then...else`) that let your program choose what to do based on whether something is TRUE or FALSE.
- **Week 8 (Loops):** Loops are for repeating code, either a set number of times (`repeat`) or forever/until a condition is met (`forever`, `repeat until`).
- **Week 10 (Debugging):** A bug is an error, and debugging is the process of finding and fixing it.

Connecting the Dots:
Look back at the "Cat and Mouse" game from Week 9. It used all these concepts together:

- We made an **algorithm** (plan).
- We used a **variable** for the score (optional).
- We used a **loop** (`forever`) to keep the game running.
- We used a **condition** (`if...then`) to check for the cat touching the mouse.
- When the game didn't work, we used **debugging** to fix it.

All five concepts work together to create a single program.

Evaluation:

This week's evaluation will be the Second Term Examination.

Assignment:

1. Study all your notes from Week 1 to Week 10 for the examination.
2. Define these five key terms: Algorithm, Variable, Condition, Loop, Debugging.
3. Which Scratch block would you use for: (a) repeating a set of blocks 10 times? (b) making a decision?
4. What is the difference between the Stage and a Sprite?
5. Why is debugging such an important skill for a programmer?

WEEK 12: Examination

Topic: Second Term Examination

Read each question carefully and do your best. Good luck!

WEEK 13: Examination

Topic: Second Term Examination (Continued)

WEEK 14: Marking and Review of Scripts

Topic: Marking and Review of Scripts

Your teacher will review the exam with you. This is a great time to learn from any mistakes and make sure you understand all the concepts from the term.

PETOA
GROUP OF SCHOOLS